

Article

DRL-Driven Intelligent SFC Deployment in MEC Workload for Dynamic IoT Networks

Seyha Ros ¹, Intae Ryoo ^{2,*} and Seokhoon Kim ^{1,3,*}¹ Department of Software Convergence, Soonchunhyang University, Asan 31538, Republic of Korea; rosseyha003@gmail.com² Department of Computer Engineering, Kyung Hee University, Yongin-si 17104, Republic of Korea³ Department of Computer Software Engineering, Soonchunhyang University, Asan 31538, Republic of Korea

* Correspondence: itryoo@khu.ac.kr (I.R.); seokhoon@sch.ac.kr (S.K.)

Abstract

The rapid increase in the deployment of Internet of Things (IoT) sensor networks has led to an exponential growth in data generation and an unprecedented demand for efficient resource management infrastructure. Ensuring end-to-end communication across multiple heterogeneous network domains is crucial to maintaining Quality of Service (QoS) requirements, such as low latency and high computational capacity, for IoT applications. However, limited computing resources at multi-access edge computing (MEC), coupled with increasing IoT network requests during task offloading, often lead to network congestion, service latency, and inefficient resource utilization, degrading overall system performance. This paper proposes an intelligent task offloading and resource orchestration framework to address these challenges, thereby optimizing energy consumption, computational cost, network congestion, and service latency in dynamic IoT-MEC environments. The framework introduces task offloading and a dynamic resource orchestration strategy, where task offloading to the MEC server ensures an efficient distribution of computation workloads. The dynamic resource orchestration process, Service Function Chaining (SFC) for Virtual Network Functions (VNFs) placement, and routing path determination optimize service execution across the network. To achieve adaptive and intelligent decision-making, the proposed approach leverages Deep Reinforcement Learning (DRL) to dynamically allocate resources and offload task execution, thereby improving overall system efficiency and addressing the optimal policy in edge computing. Deep Q-network (DQN), which is leveraged to learn an optimal network resource adjustment policy and task offloading, ensures flexible adaptation in SFC deployment evaluations. The simulation result demonstrates that the DRL-based scheme significantly outperforms the reference scheme in terms of cumulative reward, reduced service latency, lowered energy consumption, and improved delivery and throughput.

Keywords: deep reinforcement learning; multi-access edge computing; network functions virtualization; service function chaining; internet of things



Academic Editors: Bartłomiej Placzek and Marcin Bernaś

Received: 12 June 2025

Revised: 29 June 2025

Accepted: 7 July 2025

Published: 8 July 2025

Citation: Ros, S.; Ryoo, I.; Kim, S. DRL-Driven Intelligent SFC Deployment in MEC Workload for Dynamic IoT Networks. *Sensors* **2025**, *25*, 4257. <https://doi.org/10.3390/s25144257>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

1.1. Background and Motivation

The Internet of Things (IoT) has been widely used for digitalization to expose the potential of utilizing network sensing and data gathering from citizen environments [1,2]. IoT device communication with several interface technologies (i.e., Wi-Fi, Bluetooth, ZigBee,

LoRaWAN, NFC, and 6LoWPAN) through physical modulation [3–5] is used to access and communicate with network servers via fronthaul physical networks for diverse application scenarios. Thus, the increasing amount of data generated by IoT sensor networks leads to inefficient computation and resource utilization, resulting in the inability to conserve energy and reduce processing time on devices. IoT continues to generate tasks that require computing, with varying types of processing time, computing resources, and battery life. However, IoT generates massive traffic requests that often exacerbate bottlenecks and irregular fluctuations due to mobility and spatiotemporal variations in service usage patterns [6,7].

On the other hand, multi-access edge computing (MEC) can provide a solution for managing and orchestrating the resource utilization of computation from IoT sensing [8,9]. MEC has the potential to facilitate resource-intensive computation, which involves chaining virtualization resource capabilities and aims to reduce latency in communication protocols and network aspects [10–12]. However, MEC has the responsibility of addressing computational needs, leading to challenges in balancing execution delay and energy consumption. Additionally, the integration of MEC with software-defined networks/network function virtualization (SDN/NFV) controllers enhances the agility needed to reduce computation time and manage resource complexity, steering traffic patterns from ingress to egress. Service function chaining (SFC) provides instances of sequential virtual network functions (VNFs) to host virtual machines (VMs), allowing the order of VNFs to be propagated in the specified order for the required application. Consequently, MEC-enabled SDN/NFV offers lower latency and increased computational resources to manage the high demands of IoT services. Furthermore, the time needed to process resource allocation and placement for heterogeneous service functions poses a significant challenge for MEC, making real-time determination and adjustments essential for effective and efficient state transition decisions within this complex system.

To address these limitations, Deep Reinforcement Learning (DRL) has gained attention as a powerful agent for making intelligent, adaptive decisions in highly dynamic networking [13,14]. By considering various system interactions, DRL-driven approaches can continuously learn from the network environment and SFC placements [15–19]. As shown in Figure 1, leveraging the DRL-integrated MEC framework can intelligently allocate VNFs to ensure SFCs are deployed to maximize resource efficiency, minimize latency, and maintain high reliability, even amid unpredictable traffic fluctuations as the workload changes.

Furthermore, the timely and efficient advancement of MEC and SFC technologies remains challenging due to the optimal placement and resource allocation of VNFs in IoT-driven network environments. On the other hand, they have delegated the computing capacities for real-time orchestration and resource management needed to instantiate VNFs to meet the workload. Moreover, the trade-offs between critical performance metrics, such as latency, energy efficiency, cost, and resource utilization, further complicate the issue. Achieving an optimal balance between these factors is difficult due to the limited resources available at edge nodes and the unpredictable demand from IoT devices. In traditional approaches, SFC deployment often relies on predetermined rules or heuristic methods that cannot adjust efficiently in real-time, leading to inefficient resource usage and poor service quality. As the scale of IoT networks continues to grow and the demands of applications become more complex, there is a clear need for intelligent, adaptive strategies that can dynamically optimize SFC placement, resource allocation, and workload distribution across the edge network. Moreover, DRL is used to determine the policy for charging SFC deployment in the MEC server, enabling adaptive allocation of optimal decisions for instance resources in fluctuating IoT networks.

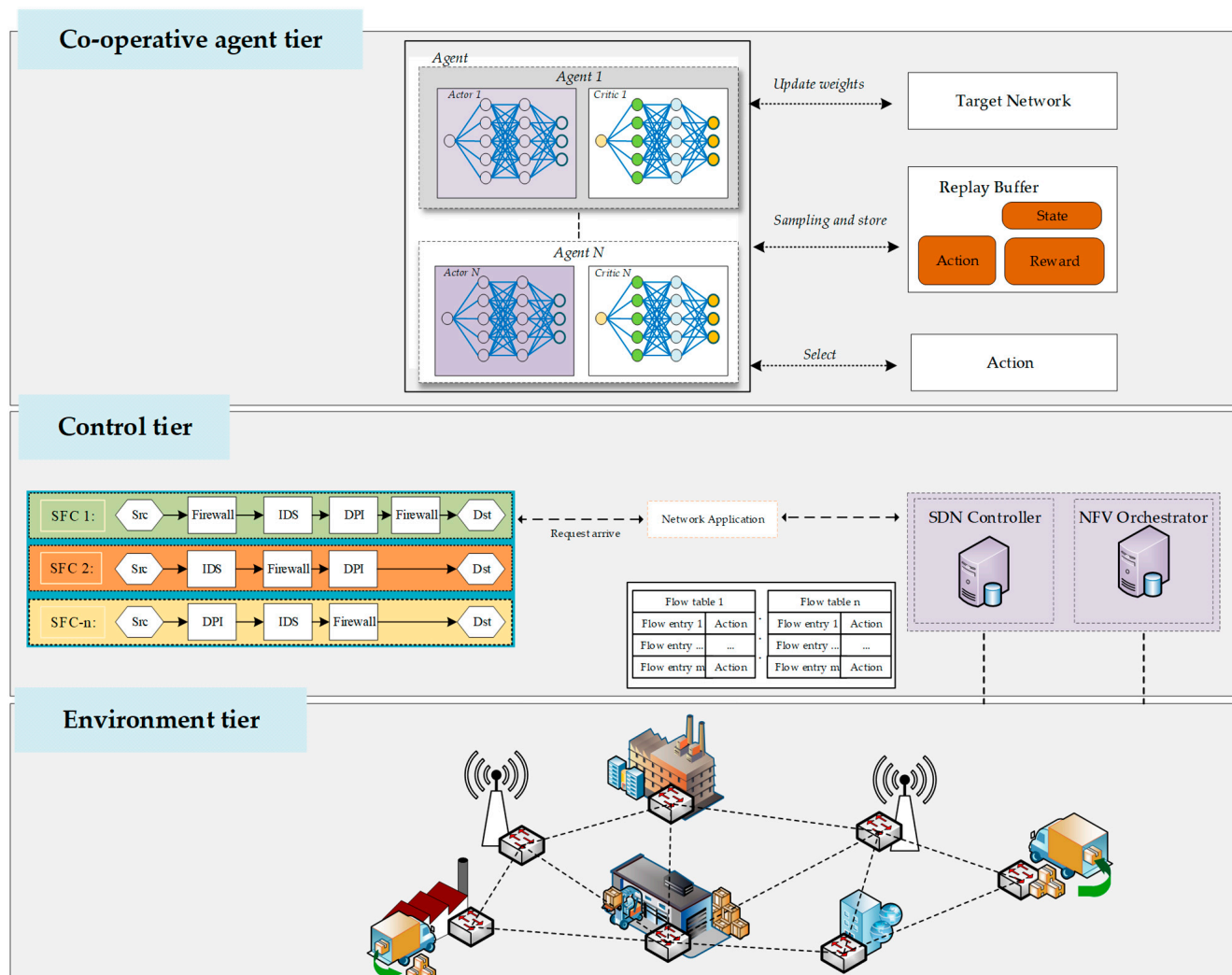


Figure 1. Multi-agent enabled in NFV for policy charging on resource utilization.

1.2. Motivation and Contributions

This paper addresses these challenges by proposing a DRL-driven framework for intelligent and efficient SFC deployment in MEC environments, capable of learning optimal placement policies that adapt to fluctuating workloads and network conditions. The following recapitalization is the main contribution of this paper:

- We formulated the VNF placement and resource allocation problem as a multi-objective optimization task, where conflicting performance goals are considered to minimize latency, conserve energy, and optimize resource utilization.
- By intelligently distributing workloads across geographically distributed edge nodes based on real-time system states, the model ensures balanced resource utilization and reduces service degradation in high-density IoT deployments. Our scheme can proactively reallocate resources to avoid and minimize overload risks, thereby improving the long-term sustainability of edge nodes.
- We design a joint optimization algorithm for task offloading and resource allocation based on the Deep Q-Network, with the optimization objectives of assisting the NFVO in instantiating resources according to the workload requirements of IoT devices.
- The reward provides comprehensive network performance. Our proposed scheme leverages DRL frameworks and the network environment to highlight significant

existing solutions across various aspects, including energy, latency, packet delivery ratio, packet drop ratio, and throughput.

1.3. Paper Organization

The remaining section of our study is organized as follows: Section 2 provides a literature review of existing work. Section 3 discusses network modeling using DRL to optimize the VNF resource allocation and placement. Section 4 outlines the performance metrics and compares the proposed methods with reference methods, concluding with a discussion in Section 5.

2. Related Work

In response to the current trend of utilizing IoT devices efficiently, many researchers have focused on resource and energy consumption to handle resource-intensive computing workloads. However, many studies investigating computing in the IoT network are still challenging due to the vast amount of big data and latency required to orchestrate resources and achieve real-time efficiency.

Many research scholars aim to optimize resource allocation and placement on MEC to enhance resource utilization concerning communication and computation-intensive resources [20–22]. Moreover, reducing latency for delay-sensitive tasks on MEC servers poses a significant challenge in this effort. Traditionally, resource allocation techniques rely on heuristic or optimization-based approaches, which struggle to adapt to the real-time and unpredictable nature of IoT traffic patterns. For example, centralized methods such as Mixed Integer Linear Programming (MILP) and convex optimization have been employed to optimize latency and bandwidth utilization [23,24]. In their work, Ref. [25] presented efficient task offloading and profit maximization in MEC-enabled 5G Internet of Vehicles (IoV). This study proposed a Lyapunov-Based Profit Maximization (LBPM) algorithm to optimize the time-averaged profit of MEC providers while ensuring timely and effective task execution from vehicles.

Additionally, the previous study on resource management in VNF placement still faces the problem of duplicates in the form of chaining and overwhelming resource utilization. For instance, refs. [26–28] study VNF placement and traffic routing simultaneously, dividing the methods into two phases: first, VNF placement, and then establishing the executing links. To achieve optimal resource utilization, ref. [27] consider the tradeoff between resource consumption and links, providing a two-phase VNF placement methodology that uses constrained depth-first search algorithms (CDFSAs) and path-based greedy algorithms to assign VNFs with minimum resource consumption [28]. The authors proposed two methods, hybrid SFC and a heuristic algorithm, to solve the dynamic SFC embedding for chaining VNF nodes. However, their utilization of the shortest path or greedy algorithms could not achieve network load balancing.

In recent years, some researchers have started applying machine learning to tackle various optimization problems [29–31], with a focus on resource allocation, resource placement, scheduling, traffic routing, resource offloading, and SFC orchestration. For instance, ref. [32] proposed PPO-ERA to provide a real-time, adaptive, and dynamic strategy for VNFs, addressing task delays and resource utilization. Meanwhile, SFC deployment over MEC servers shares idle computing resources that utilize the same SFC. In [33], VNF co-operative scheduling with priority-weighted delay is examined. Their work incorporated DRL with a target Q-network to enhance solutions for the optimal problem related to VNF scheduling, while supporting multidimensional resources in edge nodes. In [34], a multi-objective SFC mapping technique based on DQN is proposed to achieve delay and load balancing targets. In their work, Ref. [35] leverages the DQN and MQDR-based

method for the dynamic deployment of customized SFCs, including dynamic adjustments of SFCRs. Furthermore, in [36], the VNF formulation problem is addressed using integer nonlinear programming to minimize bandwidth costs and ensure service execution for end-to-end delays.

Through their strategies and significant efforts, they enhance resource efficiency and effective resource utilization. However, computation and resource allocation to idle VNFs present challenges and drawbacks, particularly in terms of reallocating resources and placement. This can lead to overwhelming utilization in real time within the same SFC.

To clarify the significance of our research, we present the utilization of VNF resource placement and allocation approaches compared to the previous studies mentioned above. We develop a novel dynamic resource allocation and placement based on diverse SFR in NFVO. VNFs are monitored by NFVI to control resource adjustments that respond to real-time changes in VNFs within SFCs, coordinating with DQN variance methods. Furthermore, we consider the priority of SFC for managing resources based on workload in computing tasks.

3. Model and Problem Formulation

This system model section discusses our system simulation and represents the complete VNF and SFC related to MEC servers and the IoT sensing network. Regarding the resource allocation issue in the VNF of the SFC, the optimization goal of this paper is to address resource placement and facilitate cost-effective application deployments by considering the services required for managing end devices.

3.1. Network Model

In the network section, we examined the allocation and placement of NFV resource utilization to align with the computation and communication demands of MEC workloads. In this paper, we illustrate the physical infrastructure resource instantiation involved in creating several Network Functions Requests (NFRs) to support multiple Service Functions (SFs) in terms of requirements and monitoring resource workload. IoT contributes to resource-intensive network data traffic, which escalates resource-heavy computation and complex workloads in edge networks, as depicted in Table 1.

In the system model, the undirected graph is leveraged to represent network function resources, where G is a set of MEC servers and V is the subset of MEC nodes that connect to the link denoted as E . The task process can be defined in the primary phase as follows:

- Offloading task from IoT to MEC server for computing task: In each *timeslot*- t , IoT devices offload *task* - j to MEC - m . Equation (1) indicates the communication model that is associated between end-devices and the MEC-server for data rate, denoted as ED_{n-m}^t for the state of allocated bandwidth $bw_{n \rightarrow m}^t$, channel gain $g_{n \rightarrow m}^t$, transmission power p_n^t , and noise $\forall \mathbb{N}$.

$$ED_{n-m}^t = bw_{n \rightarrow m}^t \log_2 \left(1 + \frac{p_n^t g_{n \rightarrow m}^t}{\forall \mathbb{N}} \right) \quad (1)$$

In this scenario, we assume that the IoT fully offloads the *task* - j to MEC - m for computation in every *timeslot* - t , and all the MEC servers are supposedly equipped in every single device.

Table 1. Notation system model in the network model.

Notation	Description
G	Set of MEC servers
E	Set of links
V	Set of VNFs
J	A set of number tasks
N	The set of IoT devices
T	Number of timeslots
$\mathcal{R}_m$	Resources of MEC servers
P_j	Set all the VNFs by $IoT-R_j$
$C_{up,m}$	Upper-bound capacity of MEC servers on (CPU, RAM, Disk)
BW	Maximum of Bandwidths
d_e	Propagation delay between nodes on <i>link e</i>
F	Number of service requests
$BW_{up,m}^t$	Upper-bound bandwidth of <i>MEC-m</i> at <i>timeslot-t</i>
$C_{re,m}^t$	Remaining resources of <i>MEC-m</i> at <i>timeslot-t</i>
	Decision variable
$x_{j,f}^v(t)$	Equals one if the VNF of the service request that is deployed at <i>node-v</i> at <i>timeslot-t</i> , otherwise
$y_j^e(t)$	Selecting the virtual link between <i>node-j</i> at <i>timeslot-t</i> , otherwise

3.2. SFC Requests

The set IoT-R is denoted by F . The j – th IoT denotes $IoT-R_j$, responding to a 5-tuple $(S_j, D_j, P_j, R_{bw,j}, R_{delay,j}, R_{cpu,j}^f)$. In this tuple, S_j and D_j represent the source IoT device and destination MEC server, respectively. The set P_j denotes the sequence of VNF requirements by $IoT-R_j$. $R_{bw,j}$ and $R_{delay,j}$ denote the bandwidth consumption required by $IoT-R_j$ and maximum tolerated E2E delay of $IoT-R_j$, respectively.

3.2.1. Resource Constraints

We ensure sufficient MEC servers' resource to host VNFs and bandwidth for handling all the $IoT-R_j$:

$$\sum_{i \in J} \sum_{f \in F_j} R_{cpu,j}^f \cdot x_{j,f}^m \leq C_{re,m}^t \quad \forall m \in G, \forall t \in T \quad (2)$$

where $R_{cpu,j}^f$ indicates CPU demand of VNF- f in request- j , and $C_{re,m}^t$ demonstrates the remaining resource capacity of MEC- m at time- t .

$$\sum_{i \in J} y_j^e(t) \cdot R_{bw,j} \leq C_{re,m}^t \quad (3)$$

where $y_j^e(t)$ indicates whether link e is traversed by IoT-R in request- $j \in R$ and equals 1 if the traffic of IoT-R travers physical link; 0 otherwise.

3.2.2. Delay Constraint

We use $\mathcal{PD}_j$ to indicate the total propagation delay of IoT-R not exceeding the maximum total of service requests:

$$\mathcal{PD}_j = \sum_{e \in E} y_j^e(t) \cdot d_e \leq R_{delay,j} \quad \forall j \in J, \forall t \in T \quad (4)$$

3.3. Optimization Modelling Designs

Improve resource allocation and placement of VNF over the MEC server. This paper aims to improve the system to maximize successful placement resources in service requests

while maintaining effectiveness by minimizing energy consumption and delay for each application under computational and networking resources constraints.

$$\text{Max} \sum_{t=1}^T \sum_{j=J} \left(\sum_{f \in F_j} \sum_{v \in V} x_{j,f}^v(t) - \lambda_1 \cdot E_j - \lambda_2 \cdot \mathcal{P}D_j \right) \quad (5)$$

s.t. (2)–(4)

3.3.1. State Space

To address the network formulation for resource utilization in orchestration and management, the state is gathered from network components with critical states, as shown in Figure 2. The process proceeds by computing resource allocation, gathering resources from the network, and orchestrating for SFR. The Markov decision process is utilized to interact within the NFVO environment.

- $\mathcal{R}_m$ represents the coordination of resources in the MEC server to preserve the ability to handle the computation tasks during IoT and offload the tasks to MEC-m.
- $C_{up,m}$ represents the upper-bound resource utilization in MEC-m at timeslot-t.
- BW is communication from the local device to MEC – m, state of total bandwidth, observed from a total bandwidth allocation and channel gain between environments.
- $BW_{up,m}^t$ represents how much bandwidth is allocated between nodes to MEC – m during the paging tasks at timeslot – t.
- A subset of a tuple $j_{j-m}^{comp}(t)$, which consists of processing tasks, including the consumed resources/energy, and time spent from experience.

$$S_t = \left\{ \mathcal{R}_m, C_{up,m}, BW, BW_{up,m}^t, j_{i-m}^{comp}(t) \right\} \quad (6)$$

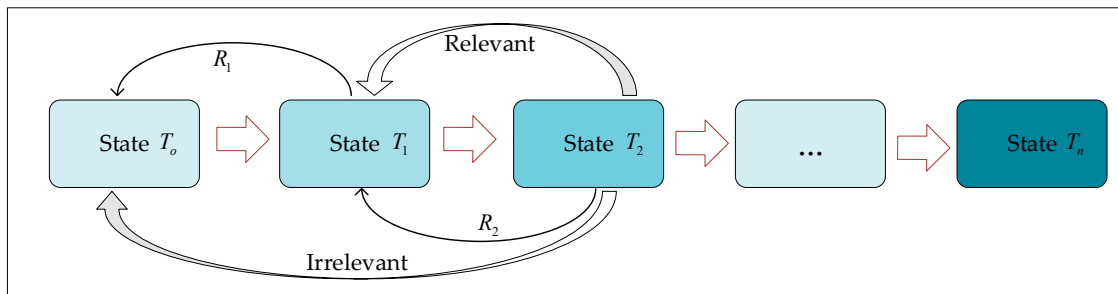


Figure 2. State space flow execution on an interval.

3.3.2. Action Space

The offloading task from IoT devices to the MEC server, whose backbone SDN controller has a global view of the flow entity, consists of the routing table synchronized as an action from the agent space. On the other hand, the mechanism of SDN-MEC enables DRL to empathize with the experienced batch of resource allocation and resource placement for performance patterns in each observation state iteration. $a_j^{off}(t)$ is set to determine the connection between IoT devices and MEC servers for offloading tasks. Additionally, $a_{mec|F}^{Plac}(t)$ is the optimal MEC selection for VNF placement to align the ordering of SFC forms in computing the task based on SFR. The corporation proposed by the agent and SDN controller is to optimize the flow rule of routing execution in installation and alleviation for the effective execution of tasks in terms of completion time within maximum tolerance delays.

$$a_t = \left\{ a_j^{off}(t), a_{mec|F}^{Plac}(t) \right\} \quad (7)$$

3.3.3. Reward Based on Policy Charging Selection

In this proposed SDN-NFV, DRL aims to deal with the efficiency of applying the action a_t into the MEC-IoT environment state s_t by gaining the reward r_t to show the impact of the action that transitions the following states: s_{t+1} . In this study, the primary reward by denoted R_{env}^t is to sum the total of two sub-rewards, reduced service latency and lowered energy consumption, denoted as r_{Lat}^t and r_{Ene}^t , respectively.

$$R_{env}^t = \lambda_{lat}(t) \cdot r_{Lat}^t + \lambda_{Ene}(t) \cdot r_{Ene}^t \quad (8)$$

where $\lambda_{lat}(t)$ and $\lambda_{Ene}(t)$ are time-varying weights ($\lambda_{lat}(t) + \lambda_{Ene}(t) = 1$) that adapt based on current conditions. When the network load is high or delay-sensitive service requests are prioritized, $\lambda_{lat}(t)$ is increased to focus on reducing latency. Conversely, under lower loads or when energy conservation is critical, $\lambda_{Ene}(t)$ is emphasized to minimize energy consumption.

The optimal policy is set to NFVO to orchestrate the resources from the overall batch that maximizes the long-term reward expectation by following Equation (9). On the other hand, Equation (10) presents an optimal policy selection for the state by following the Bellman equation.

$$\pi(t)^* = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{\pi} \left[\sum_{t \in T} \gamma^t R_{env}^t \right] \quad (9)$$

$$Q^*(s, a) = \mathbb{E}_{s'} \left[R_{env}^t + \gamma \underset{a'}{\operatorname{argmax}} Q^*(s', a') \right] \quad (10)$$

3.4. Pseudo DQN Algorithm Designs

To effectively address the dynamic and resource-constrained nature of IoT-MEC, we leverage DQN-based algorithms to ensure proper VNF placement and resource allocation utilization. In this section, Algorithm 1 presents the DQN algorithms with embedding of VNF resource placements. The DQN-based VNF placement algorithm operates in two phases: the learning phase and the execution phase. From the learning phase, input the state from the IoT-MEC environment, such as $(V, E, \mathcal{R}_m, C_{u,m}, BW, BW_{up,m}^t, C_{re,m}^t(t))$. The algorithm begins by initializing the experience replay buffer and the online and target Q-network parameters. It then initializes the system state based on incoming SFRs, which include the resource requirements and the structure of the VNFFG. A batch of candidate MEC servers is sampled without repetition for potential VNF placement. At each *timestep* – t , a random value is generated to determine the action-selection strategy. If the value is greater than a threshold ϵ -epsilon, the algorithm selects an action randomly to encourage exploration. Otherwise, it chooses the action with the maximum Q-value, computed by the online network, to exploit the learned policy. The selected action involves deploying a VNF on a chosen MEC server and computing the shortest communication path within the VNFFG. The algorithm then observes the resulting reward and transitions to the next state, storing the experience tuple as a result (s_t, a_t, r_t, s_{t+1}) in memory from which mini-batches are randomly sampled during training. For each sample, the algorithm computes a target Q-value using the Bellman equation and updates the network parameters by minimizing the mean squared error between predicted and target Q-values. The network parameter update process includes both the online Q-network update via gradient descent and the periodic synchronization of the target network to ensure stable learning. Furthermore, time complexity, training involves the sampling of a batch of size N , with each forward and backward pass costing Ψ , yielding $\mathcal{O}(N \cdot \Psi)$ per training step; the inference, which only required going forward through the network, has a complexity $\mathcal{O}(\Psi)$ per decision

step. The training is looped continuously until VNFs in SFR are successfully placed in order of requirement. In the execution phase, the trained Q-network is used to make placement decisions. The action with the maximum Q-value is selected and executed at each time step, updating the placement strategy and transitioning the environment state accordingly. This process repeats until the complete VNF chain is placed, at which point the algorithm outputs the final placement strategy P_{MEC}^j . This approach enables the algorithm to make efficient and scalable placement decisions in a resource-constrained MEC-NFV/SDN, thereby optimizing system-level performance metrics such as delay, energy consumption, and load balancing.

Algorithm 1: Pseudo-code of DQN-based VNF placement algorithms

Input State = $(V, E, \mathcal{R}_m, C_{u,m}, BW, BW_{up,m}^t, C_{re,m}^t(t))$

Output VNF placement and allocation in MEC servers P_{MEC}^j

Learning process:

1 Initialize the experience reply.

2 **For each**

3 Initialize the state s of VNF from SFR

4 Sample a batch of MEC Server – m without repetition

5 For each time step – t do

6 Generate randomly value

7 **If** random value, $\geq \epsilon$ **then**

8 Randomly select an action $a(t)$

9 **Else**

10 Select of actions with the MAX Q – value

11 $a_t = \operatorname{argmax}_{a_t} Q(s_t, a_t, \theta)$ based on online_net

12 **End if**

13 Enforce the action that deploys VNF and calculates the shortest path of VNFFG

14 Calculate the reward r_t and new state s_{t+1}

15 Store the state transition sample (s_t, a_t, r_t, s_{t+1})

16 $r(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1}} Q(s_t, a_t, \theta^-)$

17 Get updating the value of $\theta^- = \theta$ for every C steps

18 **If** the VNF placement and deployment has yet completed;

19 **Then**

20 **End if**

21 **End for**

Execution process:

22 Read the online_net and target_net

23 **For each** step – t

24 **Do**

25 Select the action with Max Q_value;

26 Execute the action and update the placement scheme:

27 $P_{MEC}^j = s_t$

28 Update the state : $s_t = s_{t+1}$

29 **If** the VNF placement is completed, then.

30 **End if**

31 **End for**

32 **Return**

33

4. Performance Evaluation

To conduct the network simulation [11,21], the network environment utilizes the NetworkX library to create network topologies for all network components. As depicted in Table 2, all simulations are executed on a computer with an AMD Ryzen (R) 7 5700X CPU, 3.0 GHz, 32 GB (Advanced Micro Devices, Inc., Santa Clara, CA, USA), NVIDIA RTX 4080 GPU (NVIDIA, Santa Clara, CA, USA), and Python programming. A network topology is instantiated to reveal the network for setting the network topology. Once the network-tolerant delay is set, the number of IoT devices is set to 100 to be attributed across five MEC servers. We utilize the DRL agent function with the OpenAI library within the DRL framework, initialized with PyTorch.

Table 2. Parameter configuration in the network and DRL framework.

Parameters	Specifications
Hosting infrastructure	Ryzen(R) 7 5700x3d CPU @ 3.0 GHz, 32 GB, NVIDIA RTX 4080 GPU
Number of IoT	100
Number of MEC servers	5
Service request configuration	NF (types = 5) SFC type of length (2–5) Flow rate (64 Kbps–4 Mbps) Tolerable time (100–500)
Task complexity and sizes	Random set (Low, normal, high)—(256 Kbits, 512 Kbits, 1024 Kbits)
Learning rate	0.001
Discount factor	0.95
Batch size	Random set (32, 64, 128, 256)
Exploration	0.5
Number of episodes	400
Python platform	PyTorch

4.1. Comparison of Proposed and Reference Schemes

Our study conducts network evaluation and sets up three comparison schemes to illustrate how the proposed approach differs from the baseline approach. This comparison illustrates the performance differences in terms of various computation resource workloads for IoT-MEC task offloading, leading to enhanced energy efficiency and reduced overhead latency.

- DQN-MIoT is our proposed method, which utilizes DRL algorithms combined with NFVO in MEC to allocate resources for VNF placement. Our proposed DRL-based NFV control leverages a DQN architecture to approximate the Q-value function, enabling collaborative configuration of SDN/NFV flow rules for optimized placement and resource allocation. This method effectively trains the function approximator to manage high-dimensional state observations and representations. As detailed in Section 3, the approach incorporates experience replay, neural network training, and synchronization with the SDN/NFV controller to learn sophisticated policies.
- DQL-MIoT is set to leverage the Deep Q-learning approach, which enables learning from the network environment and applying action to control policy; however, DL achieves low performance in the complex network topology and can only handle lightweight network topology cases.
- Greedy-MIoT represents the traditional SDN/NFV for IoT-MEC servers in computing task management. This baseline approach addresses a centralized MANO that manages resources and controls the characteristics of IoT, adhering to the definition of standard NFV rules. The resource management controller is based on topology,

traffic conditions, service requirements, and offloading policies. Greedy-MIoT relies on network-level optimization, efficient resource management, and resource allocation for non-complex applications.

- Random-MIoT indicates that the algorithm stochastically chooses an MEC service to an ingress source from traversing a VNF resource instance and a routing path to commonly chaining two adjacent VNF instances for every incoming IoT-R.

4.2. Results and Discussion

In this section, we present the results for the proposed DQN-MIoT and reference schemes, namely DQL-MIoT, Greedy-MIoT, and Random-MIoT, in terms of cumulative reward, sub-rewards for latency and energy, packet drop ratio, packet delivery ratio, throughput, and delay in services. The measurement of this performance metric significantly showcases how our system setting integrates with the DQN framework, ensuring the reliability of chaining VNFs and adjusting for resource differences under diverse applications and workloads. The same topology and task size are used in this simulation, but the controller and agent differ in terms of performance metrics. The result demonstrates the total reward throughout each significant scheme's DQN phase for exploration and exploitation within 400 episodes. Figure 3 illustrates the results of the proposed method, showing the total reward over 400 episodes, as defined in Equation (8). During the early learning phase (episodes 0–100), all approaches begin with negative cumulative rewards due to random policy exploration and ineffective action selection. As training progresses, DQN-MIoT shows a rapid improvement, achieving a positive cumulative reward after approximately 150 episodes, steadily increasing to 32.89 at episode 400. Meanwhile, DQL-MIoT gradually improves and achieves a final cumulative reward of 19.74, which is approximately 40% lower than DQN-MIoT. The Greedy-MIoT method shows minimal improvement, ending with a cumulative reward of only 2.53, which is approximately 92% lower than that of DQN-MIoT. The Random-MIoT baseline performs the worst, consistently maintaining negative rewards and ending at -26.48 , indicating no effective learning or adaptation.

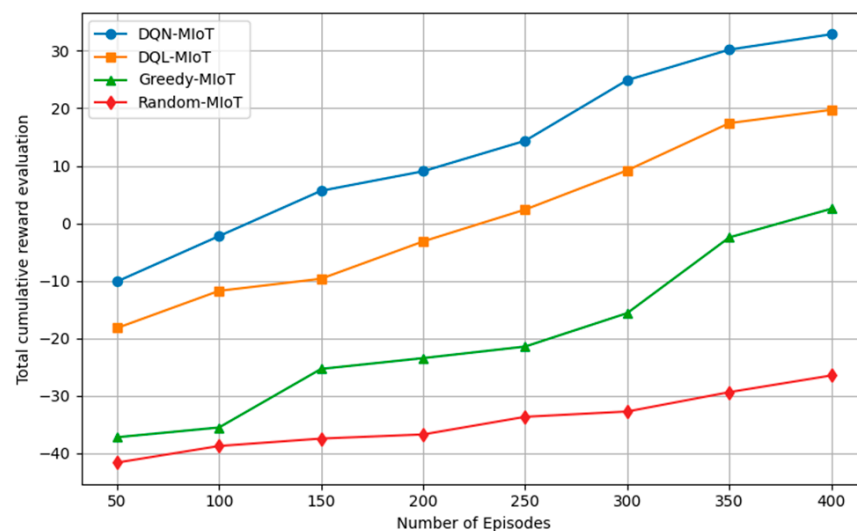


Figure 3. Result of cumulative reward evaluation.

In Figure 4a, initially, all methods exhibit negative sub-rewards, indicating poor latency performance due to random or sub-optimal policy actions during early learning phases. However, by episode 200, the DQN-MIoT approach begins to yield positive sub-reward values, signaling a successful adaptation to the MEC workload dynamics in IoT scenarios. As learning continues, DQN-MIoT and DQL-MIoT demonstrate robust

convergence behavior, with DQN-MIoT consistently achieving higher sub-reward values than DQL-MIoT. By episode 400, DQN-MIoT achieves a latency sub-reward of 29.57, compared to 26.51 for DQL-MIoT, only 5.17 for Greedy-MIoT, and -30.31 for random-MIoT. This substantial gap underscores the benefits of employing a value-based Q-value with optimized exploration strategies for latency-sensitive SFC placement. Figure 4b illustrates the efficiency of energy score points as DQN-MIoT's training process transitions the agent into a more stable policy regime, resulting in a higher immediate reward score. DQN-MIoT demonstrated a steady improvement, eventually peaking as the training progressed due to its adaptive learning strategy, which optimized energy consumption under network constraints. In contrast, DQL-MIoT showed more gradual progress, reflecting its slower convergence in handling energy efficiency within the MEC-enabled IoT environment. The Greedy-MIoT and Random-MIoT consistently underperformed with negative energy sub-rewards throughout most episodes, as they lacked adaptive mechanisms to manage resource allocation effectively. From the reward efficiency perspective, the proposed DQN-MIoT scheme dynamically adjusts resource allocation and offloading decisions, effectively balancing computational capacity and energy constraints. This adaptability enables improved handling of resource-constrained and computation-intensive tasks within the network, demonstrating strong suitability for MEC-enabled IoT scenarios where energy efficiency and task completion are crucial.

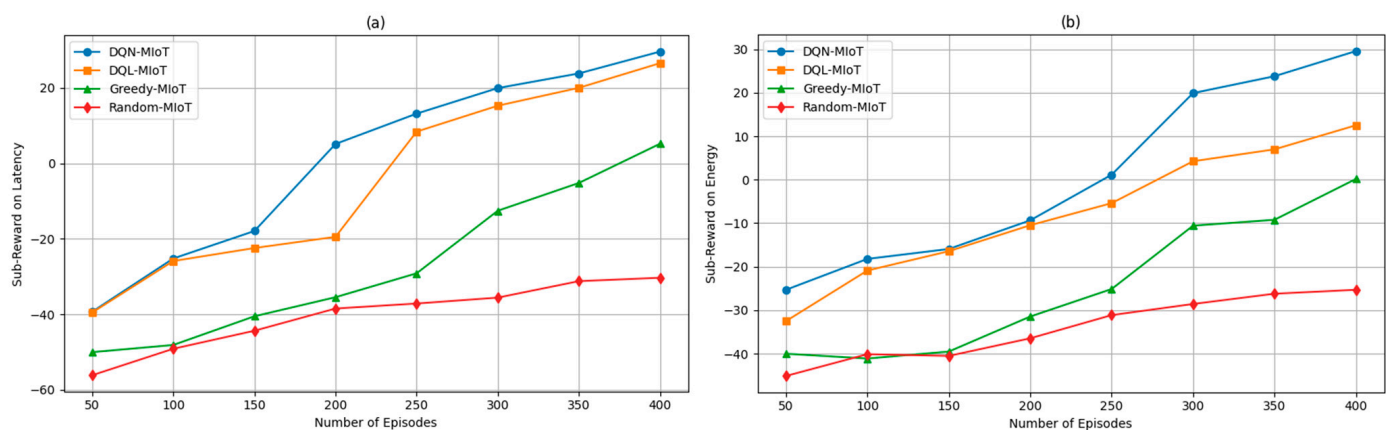


Figure 4. Performance metrics on sub-rewards of (a) latency and (b) energy.

The results are demonstrated in Figure 5a,b for task delivery and drop ratios, where network states are configured consecutively by setting different network conditions, congestion levels, and application tasks every 400 simulation intervals. We evaluate the agent based on the fluctuation of investment results in the network parameter setting compared to the diversity of state observation. In high traffic fluctuations and computation-intensive environments, DQN-MIoT maintains its advantage with a PDR of 99.9681%, whereas DQL-MIoT drops to 99.9145%, and Greedy-MIoT declines further to 99.7568%. These results demonstrate the robustness and reliability of the DQN-MIoT framework in maintaining high data delivery rates even under increasing network loads. Meanwhile, at shorter simulation durations (20 s), DQN-MIoT achieves a notably low packet drop ratio of 0.05%, outperforming DQL-MIoT (0.09%), Greedy-MIoT (0.14%), and Random-MIoT (0.24%). As the simulation time increases to 380 s, DQN-MIoT maintains its performance advantage, recording a drop ratio of 0.18%, while DQL-MIoT, Greedy-MIoT, and random-MIoT reach 0.245%, 0.3401%, and 0.3513%, respectively. This consistent performance trend underscores the superior decision-making and adaptability of the DQN-MIoT framework in managing network traffic, confirming its suitability for real-time, data-sensitive applications.

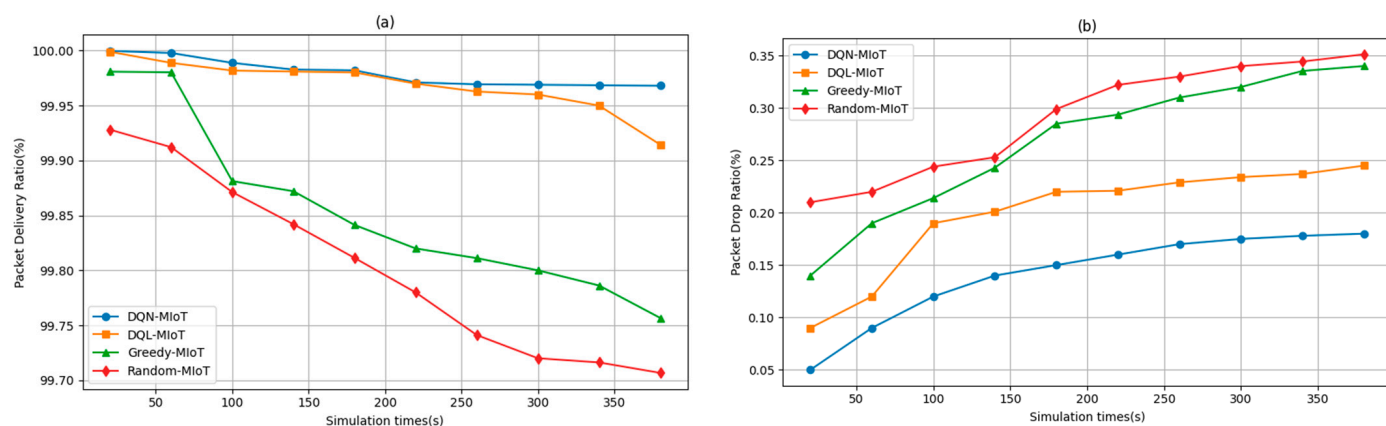


Figure 5. Performance metrics of (a) packet delivery and (b) packet drop ratio.

The throughput results presented in Figure 6, across different simulation times, further reinforce the superior performance of the DQN-MIoT approach compared to DQL-MIoT and Greedy-MIoT. At 20 s, DQN-MIoT achieves the highest throughput of 805.94 units, slightly surpassing DQL-MIoT (803.12) and notably outperforming Greedy-MIoT (799.28). DQN-MIoT maintains a stable throughput at 801.75, while DQL-MIoT, Greedy-MIoT, and random-MIoT drop to 798.11, 780.82, and 772.63, respectively. This steady trend indicates the DQN-MIoT model's efficiency in handling data traffic under varying load conditions.

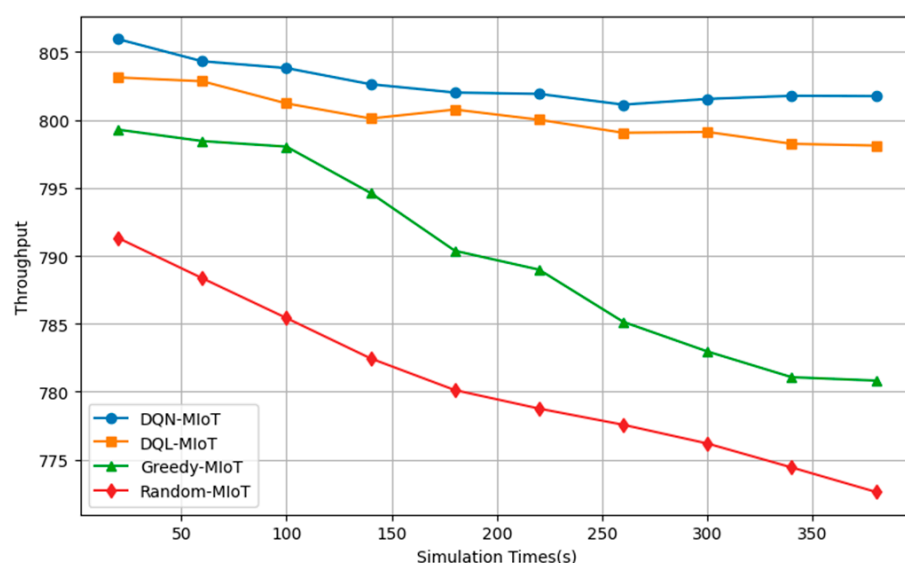


Figure 6. Result on throughput.

A thorough evaluation of performance metrics reveals that DQN-MIoT significantly outperforms reference schemes in terms of convergence speed, final reward value, and overall learning efficiency. Specifically, DQN-MIoT achieves a 66.7% higher final cumulative reward than DQL-MIoT, over 12 times higher than Greedy-MIoT, and vastly superior performance compared to Random-MIoT. This substantial improvement in DQN-MIoT's strong ability to learn effective policies for balancing latency and energy trade-offs in dynamic MEC-enabled IoT environments leads to more intelligent and robust SFC deployment.

5. Conclusions

In this paper, we proposed the DQN-based NFV in IoT-MEC to handle resource computation for diverse services. We designed an intelligent task offloading and resource orchestration framework for dynamic IoT-MEC environments. The framework integrates SFC for VNF placement, dynamic route path selection, and workload distribution through MEC-based task offloading. By leveraging DRL, specifically a DQN-based Markov Decision Process, the system learns to make adaptive decisions for task offloading and resource allocation. Our simulation results indicate that DQN-MIoT outperforms baseline approaches in terms of sub-rewards on latency, energy consumption for MEC workloads within heavy fluctuating traffic changes, packet delivery ratio, packet drop ratio, and throughput in various network conditions and resource allocation values. In the future, we aim to incorporate memory-augmented and graph-enhanced learning models to overcome the limitations of LLMs in capturing long-term, multi-stage attack patterns. Moreover, future simulations will consider federated and edge-aware deployments to handle resource constraints in real-time environments. We will explore lightweight, explainable AI (XAI)-driven frameworks to improve interpretability and transparency in task orchestration.

Author Contributions: Conceptualization, S.R., I.R. and S.K.; methodology, S.R.; software, S.R.; validation, S.R.; formal analysis, S.R.; investigation, S.K.; resources, S.K.; data curation, S.R.; writing—original draft preparation, S.R. and S.K.; writing—review and editing, S.R.; visualization, S.R.; supervision, I.R. and S.K.; project administration, S.K.; funding acquisition, S.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant, funded by the Korean government (MSIT) (No. RS-2022-00167197, Development of Intelligent 5G/6G Infrastructure Technology for The Smart City); in part by BK21 FOUR (Fostering Outstanding Universities for Research) under Grant 5199990914048; and in part by the Soonchunhyang University Research Fund.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Derived data supporting the findings of this study are available from the corresponding author on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Anitha, P.; Vimala, H.S.; Shreyas, J. Comprehensive Review on Congestion Detection, Alleviation, and Control for IoT Networks. *J. Netw. Comput. Appl.* **2024**, *221*, 103749. [\[CrossRef\]](#)
2. Alsharif, M.H.; Kelechi, A.H.; Jahid, A.; Kannadasan, R.; Singla, M.K.; Gupta, J.; Geem, Z.W. A Comprehensive Survey of Energy-Efficient Computing to Enable Sustainable Massive IoT Networks. *Alex. Eng. J.* **2024**, *91*, 12–29. [\[CrossRef\]](#)
3. Ma, H.; Tao, Y.; Fang, Y.; Chen, P.; Li, Y. Multi-Carrier Initial-Condition-Index-Aided DCSK Scheme: An Efficient Solution for Multipath Fading Channel. *IEEE Trans. Veh. Technol.* **2025**, 1–14. [\[CrossRef\]](#)
4. Yu, Q.; Wang, H.; He, D.; Lu, Z. Enhanced Group-Based Chirp Spread Spectrum Modulation: Design and Performance Analysis. *IEEE Internet Things J.* **2025**, *12*, 5079–5092. [\[CrossRef\]](#)
5. Li, J.; Sun, G.; Wu, Q.; Niyato, D.; Kang, J.; Jamalipour, A.; Leung, V.C.M. Collaborative Ground-Space Communications via Evolutionary Multi-Objective Deep Reinforcement Learning. *IEEE J. Sel. Areas Commun.* **2024**, *42*, 3395–3411. [\[CrossRef\]](#)
6. Mijumbi, R.; Serrat, J.; Gorricho, J.-L.; Bouten, N.; De Turck, F.; Boutaba, R. Network Function Virtualization: State-of-The-Art and Research Challenges. *IEEE Commun. Surv. Tutor.* **2016**, *18*, 236–262. [\[CrossRef\]](#)
7. Aboubakar, M.; Kellil, M.; Roux, P. A Review of IoT Network Management: Current Status and Perspectives. *J. King Saud Univ. Comput. Inf. Sci.* **2021**, *34*, 4163–4176. [\[CrossRef\]](#)
8. Liyanage, M.; Porambage, P.; Ding, A.Y.; Kalla, A. Driving Forces for Multi-Access Edge Computing (MEC) IoT Integration in 5 G. *ICT Express* **2021**, *7*, 127–137. [\[CrossRef\]](#)

9. Singh, R.; Sukapuram, R.; Chakraborty, S. A Survey of Mobility-Aware Multi-Access Edge Computing: Challenges, Use Cases and Future Directions. *Ad Hoc Netw.* **2023**, *140*, 103044. [\[CrossRef\]](#)
10. Wang, H.; Chen, P. Parallelism-Aware Service Function Chain Placement for Delay-Sensitive IoT Applications with VNF Reuse in Mobile Edge Computing. In Proceedings of the 2024 IEEE International Conference on Web Services (ICWS), Shenzhen, China, 7–13 July 2024; pp. 968–973. [\[CrossRef\]](#)
11. Ros, S.; Tam, P.; Song, I.; Kang, S.; Kim, S. Handling Efficient VNF Placement with Graph-Based Reinforcement Learning for SFC Fault Tolerance. *Electronics* **2024**, *13*, 2552. [\[CrossRef\]](#)
12. Tam, P.; Kim, S. Graph-Based Deep Reinforcement Learning in Edge Cloud Virtualized O-RAN for Sharing Collaborative Learning Workloads. *IEEE Trans. Netw. Sci. Eng.* **2024**, *12*, 302–318. [\[CrossRef\]](#)
13. Wang, H.; Guo, R.; Ma, P.; Ruan, C.; Luo, X.; Ding, W.; Zhong, T.; Xu, J.; Liu, Y.; Chen, X. Towards Mobile Sensing with Event Cameras on High-Agility Resource-Constrained Devices: A Survey. *arXiv* **2025**, arXiv:2503.22943. [\[CrossRef\]](#)
14. Ullah, S.A.; Bibi, M.; Hassan, S.A.; Abou-Zeid, H.; Qureshi, H.K.; Jung, H.; Mahmood, A.; Gidlund, M.; Hossain, E. From Nodes to Roads: Surveying DRL Applications in MEC-Enhanced Terrestrial Wireless Networks. *IEEE Commun. Surv. Tutor.* **2025**, *1*–42. [\[CrossRef\]](#)
15. Ding, H.; Zhao, Z.; Zhang, H.; Liu, W.; Yuan, D. DRL-Based Computation Efficiency Maximization in MEC-Enabled Heterogeneous Networks. *IEEE Trans. Veh. Technol.* **2024**, *73*, 15739–15744. [\[CrossRef\]](#)
16. Wu, J.; Yin, J.; Zhao, X.; Liu, Y. DRL-Driven Adaptive SFC Deployment with GCN in Distributed Cloud Networks. In Proceedings of the 2025 IEEE 4th Information Technology and Mechatronics Engineering Conference (ITOEC), Chongqing, China, 14–16 March 2025; pp. 1269–1273. [\[CrossRef\]](#)
17. Onsu, M.A.; Lohan, P.; Kantarci, B.; Janulewicz, E.; Slobodrian, S. A New Realistic Platform for Benchmarking and Performance Evaluation of DRL-Driven and Reconfigurable SFC Provisioning Solutions. *arXiv* **2024**, arXiv:2406.10356. [\[CrossRef\]](#)
18. Liu, Y.; Lu, Y.; Li, X.; Qiao, W.; Li, Z.; Zhao, D. SFC Embedding Meets Machine Learning: Deep Reinforcement Learning Approaches. *IEEE Commun. Lett.* **2021**, *25*, 1926–1930. [\[CrossRef\]](#)
19. Escheikh, M.; Taktak, W. Online QoS/QoE-Driven SFC Orchestration Leveraging a DRL Approach in SDN/NFV Enabled Networks. *Wirel. Pers. Commun.* **2024**, *137*, 1511–1538. [\[CrossRef\]](#)
20. Golec, M.; Khamayseh, Y.; Melhem, S.B.; Alwarafy, A. LLM-Driven APT Detection for 6G Wireless Networks: A Systematic Review and Taxonomy. *arXiv* **2025**, arXiv:2505.18846. [\[CrossRef\]](#)
21. Ros, S.; Kang, S.; Iv, T.; Song, I.; Tam, P.; Kim, S. Priority-Aware Resource Allocation for VNF Deployment in Service Function Chains Based on Graph Reinforcement Learning. *Comput. Mater. Contin.* **2025**, *83*, 1649–1665. [\[CrossRef\]](#)
22. Xu, Y.; He, Z.; Li, K. Resource Allocation and Placement in Multi-Access Edge Computing. *Stud. Big Data* **2024**, *151*, 39–62. [\[CrossRef\]](#)
23. Tomassilli, A.; Giroire, F.; Huin, N.; Pérennès, S. rovably Efficient Algorithms for Placement of Service Function Chains with Ordering Constraints. In Proceedings of the IEEE INFOCOM 2018—IEEE Conference on Computer Communications, Honolulu, HI, USA, 16–19 April 2018. [\[CrossRef\]](#)
24. Guérout, T.; Gaoua, Y.; Artigues, C.; Costa, G.D.; Lopez, P.; Monteil, T. Mixed Integer Linear Programming for Quality of Service Optimization in Clouds. *Future Gener. Comput. Syst.* **2017**, *71*, 1–17. [\[CrossRef\]](#)
25. Sun, G.; Wang, Z.; Su, H.; Yu, H.; Lei, B.; Guizani, M. Profit Maximization of Independent Task Offloading in MEC-Enabled 5G Internet of Vehicles. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 16449–16461. [\[CrossRef\]](#)
26. Pei, J.; Hong, P.; Xue, K.; Li, D. Efficiently Embedding Service Function Chains with Dynamic Virtual Network Function Placement in Geo-Distributed Cloud System. *IEEE Trans. Parallel Distrib. Syst.* **2019**, *30*, 2179–2192. [\[CrossRef\]](#)
27. Jin, P.; Fei, X.; Zhang, Q.; Liu, F.; Li, B. Latency-aware VNF Chain Deployment with Efficient Resource Reuse at Network Edge. In Proceedings of the IEEE Conference on Computer Communications (IEEE INFOCOM), Toronto, ON, Canada, 6–9 July 2020; pp. 267–276.
28. Zheng, D.; Peng, C.; Liao, X.; Cao, X. Toward Optimal Hybrid Service Function Chain Embedding in Multiaccess Edge Computing. *IEEE Internet Things J.* **2020**, *7*, 6035–6045. [\[CrossRef\]](#)
29. Tam, P.; Ros, S.; Song, I.; Kang, S.; Kim, S. A Survey of Intelligent End-To-End Networking Solutions: Integrating Graph Neural Networks and Deep Reinforcement Learning Approaches. *Electronics* **2024**, *13*, 994. [\[CrossRef\]](#)
30. Mao, M.; Hong, M. YOLO Object Detection for Real-Time Fabric Defect Inspection in the Textile Industry: A Review of YOLOv1 to YOLOv11. *Sensors* **2025**, *25*, 2270. [\[CrossRef\]](#)
31. Sun, R.; Cheng, N.; Li, C.; Chen, F.; Chen, W. Knowledge-Driven Deep Learning Paradigms for Wireless Network Optimization in 6G. *IEEE Network* **2024**, *38*, 70–78. [\[CrossRef\]](#)
32. Zhai, X.; He, Z.; Xiao, Y.; Wu, J.; Yu, X. Dynamic VNF Deployment and Resource Allocation in Mobile Edge Computing. In Proceedings of the 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Kaifeng, China, 30 October–2 November 2024; pp. 573–581. [\[CrossRef\]](#)

33. Yao, J.; Wang, J.; Wang, C.; Yan, C. DRL-Based VNF Cooperative Scheduling Framework with Priority-Weighted Delay. *IEEE Trans. Mob. Comput.* **2024**, *23*, 11375–11388. [[CrossRef](#)]
34. Xu, S.; Li, Y.; Guo, S.; Lei, C.; Liu, D.; Qiu, X. Cloud-Edge Collaborative SFC Mapping for Industrial IoT Using Deep Reinforcement Learning. *IEEE Trans. Ind. Inform.* **2021**, *18*, 4158–4168. [[CrossRef](#)]
35. Zhu, R.; Wang, P.; Geng, Z.; Zhao, Y.; Yu, S. Double-Agent Reinforced VNFC Deployment in EONs for Cloud-Edge Computing. *J. Light. Technol.* **2023**, *41*, 5193–5208. [[CrossRef](#)]
36. Gao, X.; Liu, R.; Kaushik, A.; Zhang, H. Dynamic Resource Allocation for Virtual Network Function Placement in Satellite Edge Clouds. *IEEE Trans. Netw. Sci. Eng.* **2022**, *9*, 2252–2265. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.